

Q) Convert the following C code into RISC-V assembly language. Arguments **g, h, i, j** are in **x10,x11,x12,x13**. Local variables **a,b,c,d, and sum** are stored in registers **x20, x21, x22, x23, and x24**. Local variables **f** and **m** are also using registers **x20** and **x21**.

```
int main() {
    a = 10;
    b = 20;
    c = 30;
    d = 40;
    sum = sub_add(a, b, c, d);
}

int sub_add(int g, int h, int i, int j) {
    int f = (g + h);
    int m = (i + j);
    int x = f - m;
    return x;
}
```

Solution:

Main:

```
addi x20, x0, 10
addi x21, x0, 20
addi x22, x0, 30
addi x23, x0, 40
add x10, x0, x20 # x10 = 10 (acquire storage for argument g)
addi x11, x0, 20 # x11 = 20 (acquire storage for argument h)
addi x12, x0, 30 # x12 = 30 (acquire storage for argument i)
addi x13, x0, 40 # x13 = 40 (acquire storage for argument j)
jal x1, sub_add # jump to sub_add, save return address in x1
addi x24, x10, 0 # store the return value in x10 to sum.
beq x0, x0, exit
```

sub_add:

```
addi x2, x2, -16    # make space on stack to save 2 registers (X20 and X21)
sd  x20, 8(x2)      # save x20 to stack
sd  x21, 0(sp)      # save x21 to stack
```

```
add  x20, x10, x11   # f = g + h
add  x21, x12, x13   # m = i + j
sub  x5, x20, x21     # x = f - m
```

```
addi x10, x5, 0      # save the return value in x10
```

```
ld  x21, 0(sp)       # restore x21 from stack
ld  x20, 8(sp)        # restore x20 from stack
```

```
addi sp, sp, 16      # deallocate 16 bytes from stack
jalr x0, 0(x1)        # return to caller (jump to address in x1)
```

exit: